



L3 informatique

Course Notes
Software Engineering

Course 3
Modeling
Package diagram

Presented by
Mr KHELIFA N.

Introduction

- The visualization, construction, specification, and documentation of large-scale systems involve the management of a substantial number of classes, interfaces, components, nodes, diagrams, and other modeling elements.
- This necessitates the organization of these elements into coherent groups.
- A **package** is a generic mechanism used to structure modeling elements into such groups.
- The decomposition into packages should reflect a logical partitioning of the system under development, corresponding to homogeneous namespaces.
- It is possible to control the visibility of elements in order to present the different architectural views of a system.

Package Diagram

- It is a **structural diagram** and includes the following elements:
 - **Package**
 - **Relationships between packages**
(dependencies)

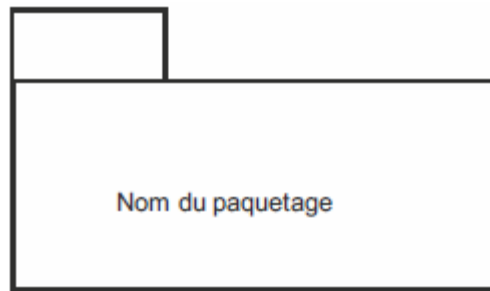
Package

- A **namespace** is defined as a context that provides a scope for identifiers.
- A **package** groups modeling elements, also called **members** (including classes, diagrams, functions, interfaces, etc.), that pertain to a subset of the system.
- The elements within a package can have declared visibility, either **public (+)** or **private (-)**.
- A package can **import** elements from another package.
- A package can also be **merged** with another package.

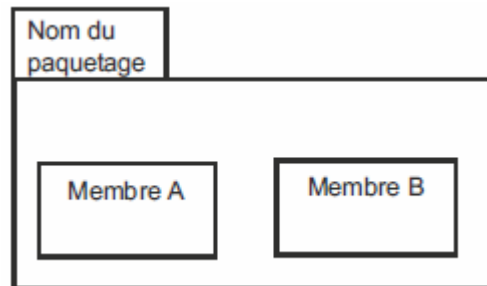
Representation

- A package can be represented in **three possible ways**:

Global representation – The name of the package is placed inside the large rectangle.

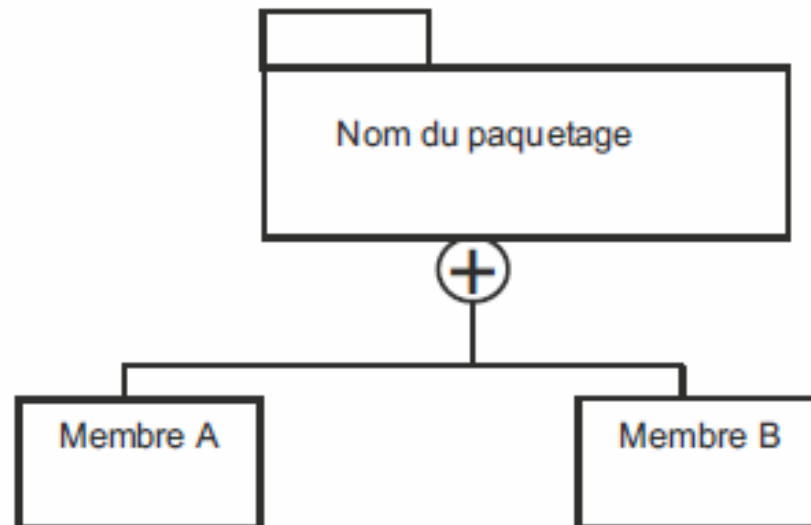


Detailed representation – The members of the package are shown, and the name of the overall package is placed in the small rectangle.



Representation

- **Expanded (or exploded) representation**
 - The members of the package are connected by a link to the package using the « package » symbol : $\oplus$.

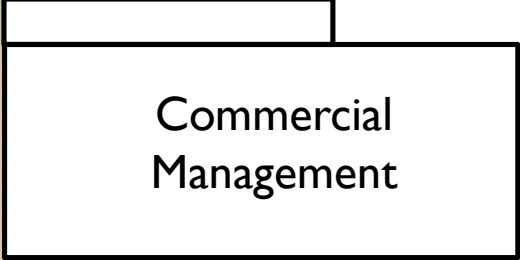


Exemple

- The **Commercial Management** package, which contains two members: **Order** and **Invoice**.

Example

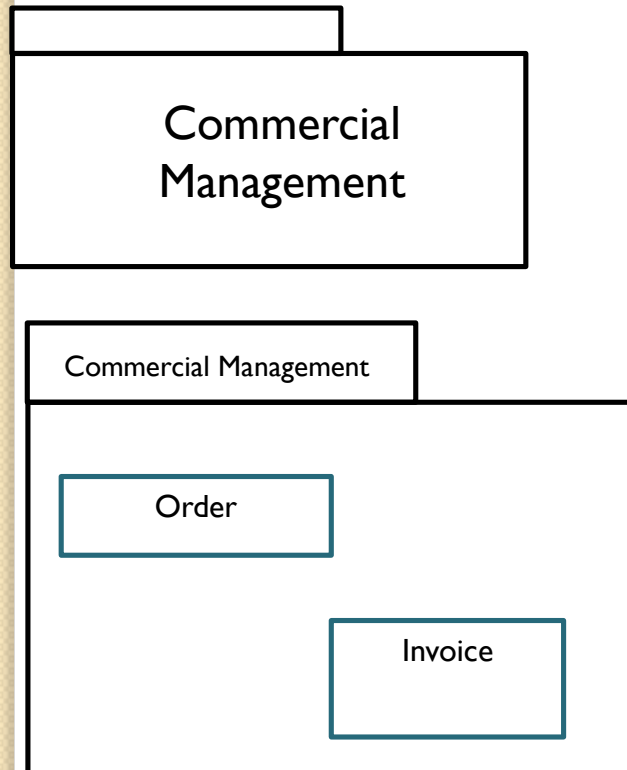
- The **Commercial Management** package, which contains two members: **Order** and **Invoice**.



Commercial
Management

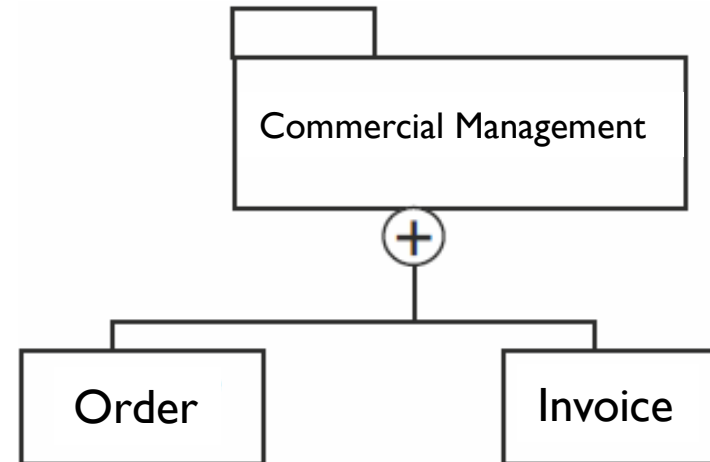
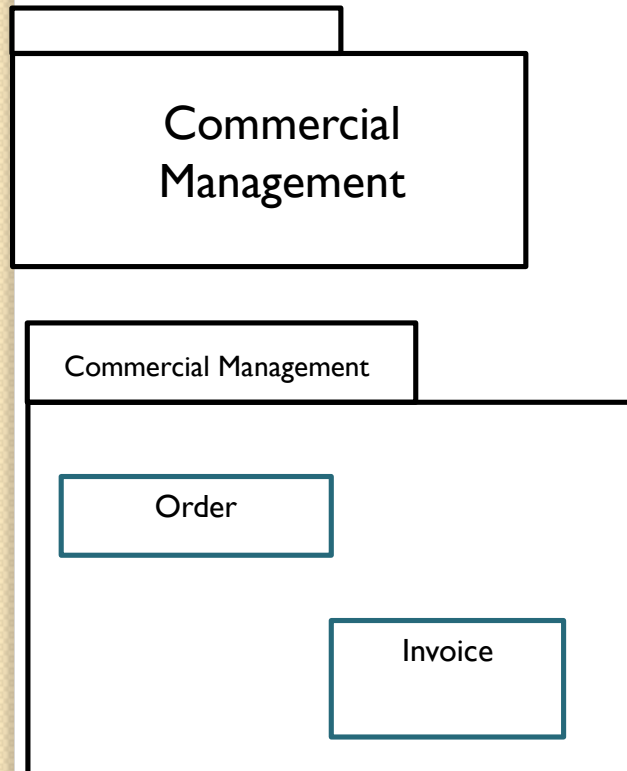
Example

- The **Commercial Management** package, which contains two members: **Order** and **Invoice**.



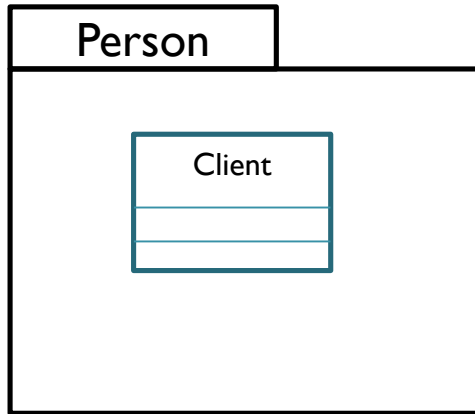
Example

- The **Commercial Management** package, which contains two members: **Order** and **Invoice**.

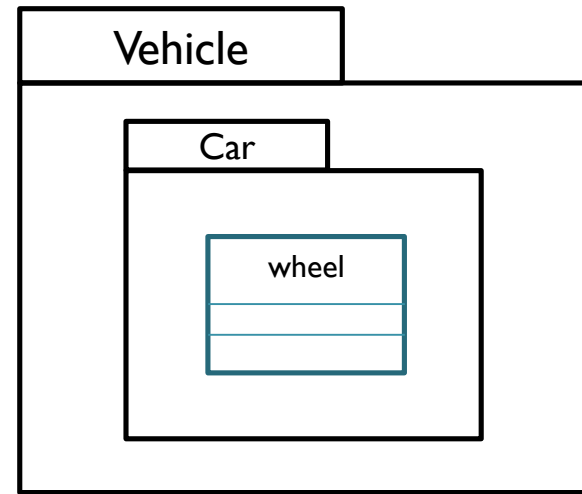


Namespaces of the members

- To reference an element within a package, you must specify the **package name** (namespace) followed by **double colons (::)**, and then the name of the element.



Person::Client



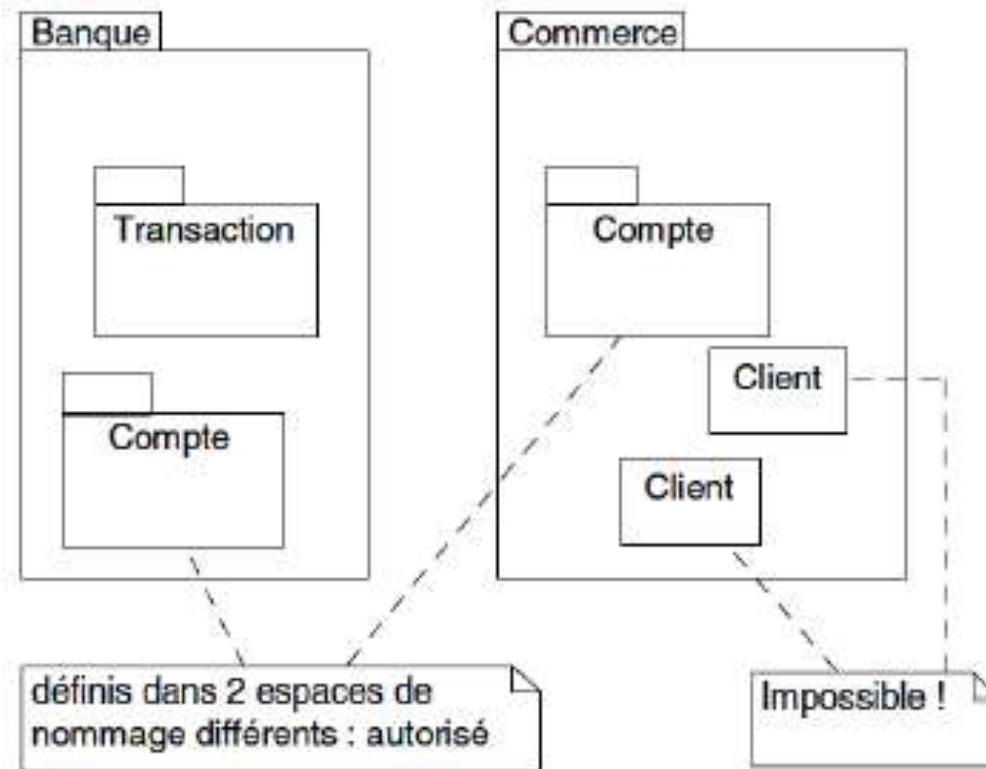
Vehicle::Car::wheel

Packages can be **nested** (hierarchical decomposition) but cannot **overlap**.

An element of the system can belong to **only one package**.

Each package must have a **unique name**.

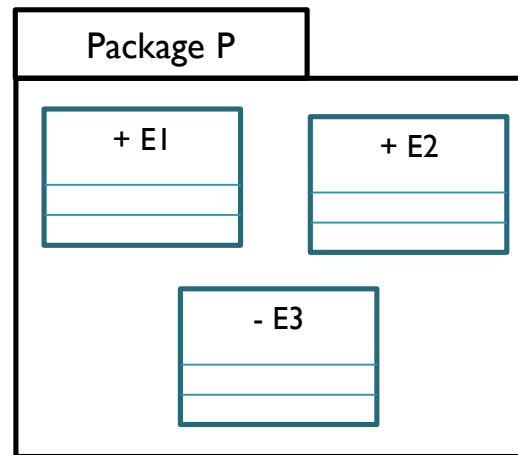
Exemple



- **Full name** = the name prefixed by the names of the enclosing packages:
Bank::Account $\neq$ Commerce::Account

Visibility

- **Visibility of an element E within a package P:**
- **Public (+):** The element is visible to all (accessed using its full name).
- **Private (-):** The element is visible only to:
 - Other elements within P
 - Elements contained within E (if E is a package)

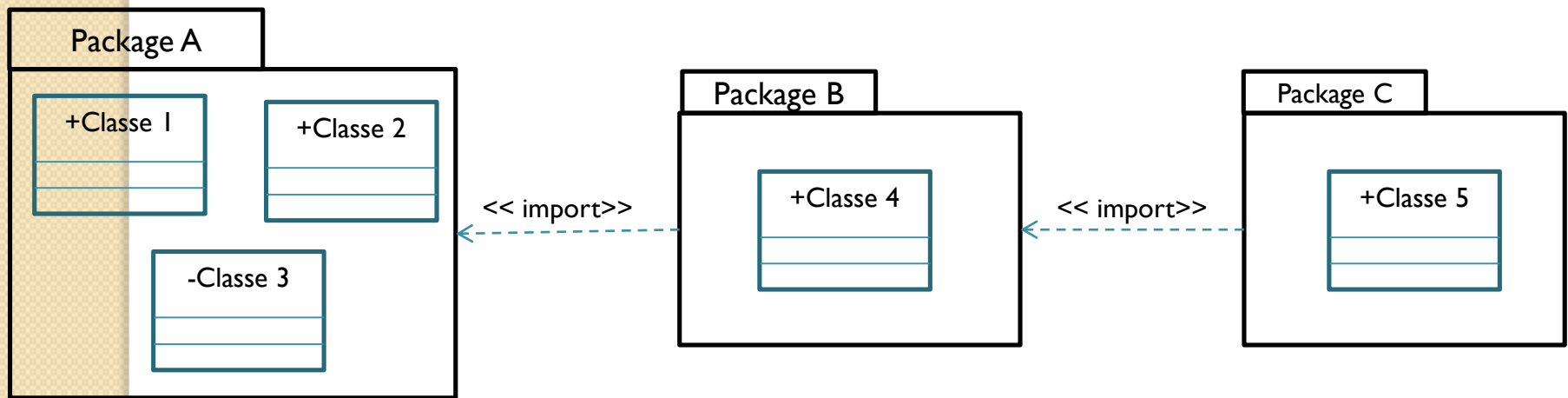


Dependency between packages

- **Dependencies between packages** can be qualified by a **visibility level**, which can be either **public** or **private**.
By default, the visibility type is **public**.
Each visibility type is associated with a **dependency link**.
- The three types of package dependencies are:
 - « **import** »
 - « **access** »
 - « **merge** »

Import

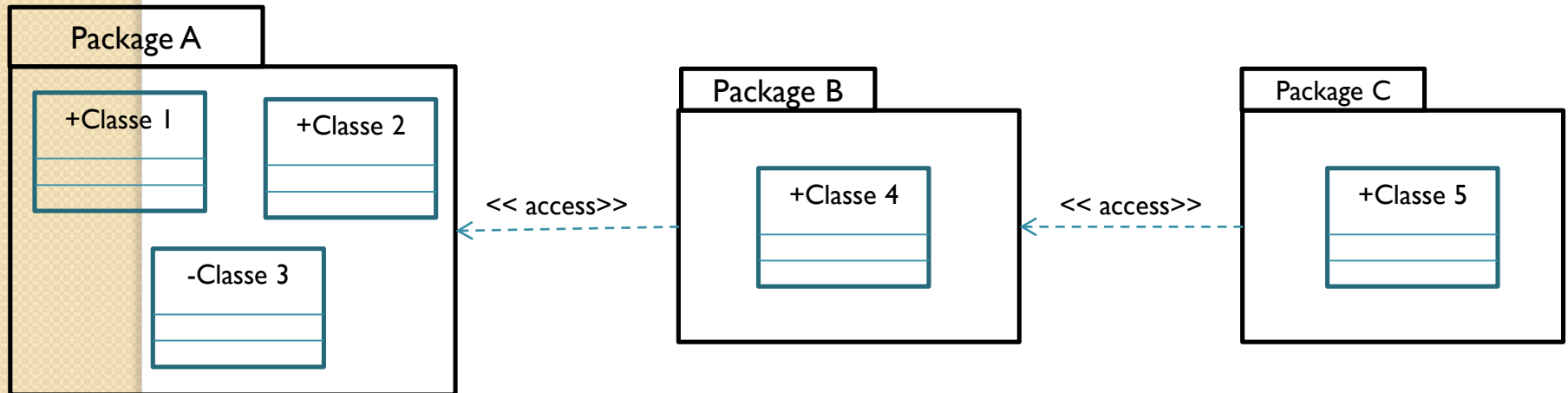
- This type of dependency allows a given package to **import the namespace** of another package. All members of the given package have access to all the names of the members of the imported package **without needing to explicitly use the name of the imported package**. This type of dependency corresponds to a link with **public visibility**.



Package **B** imports **Class I** and **Class2** (but not **Class3**, which has **private visibility**).
Class I and **Class2** have **public visibility** within package B.
Package **C** imports **Class I**, **Class2**, and **Class4**.

Access

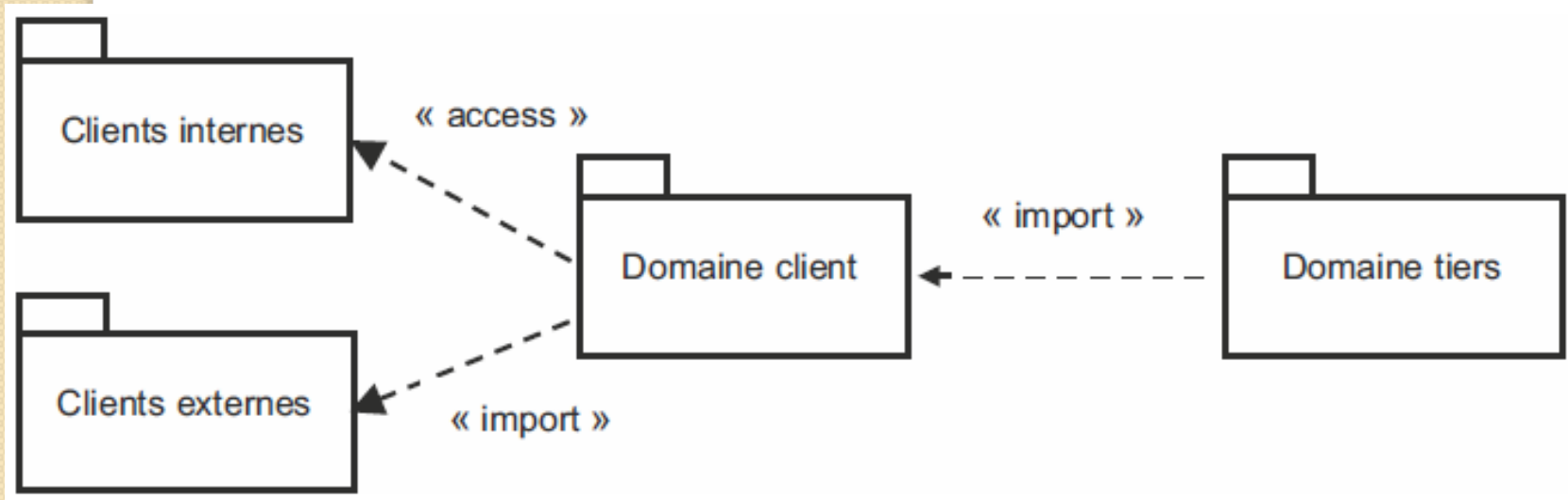
- This type of dependency allows a given package to **access the namespace** of a target package. The namespace is therefore **not imported** and **cannot be transmitted to other packages by transitivity**. This type of dependency corresponds to a link with **private visibility**.



Package **B** has access to **Class 1** and **Class 2** (but not **Class 3**, which has **private visibility**). **Class 1** and **Class 2** have **private visibility** within package B.

Package **C** has access to **Class 4** (but not to **Class 1** and **Class 2**, which have **private visibility** within package B).

Exemple



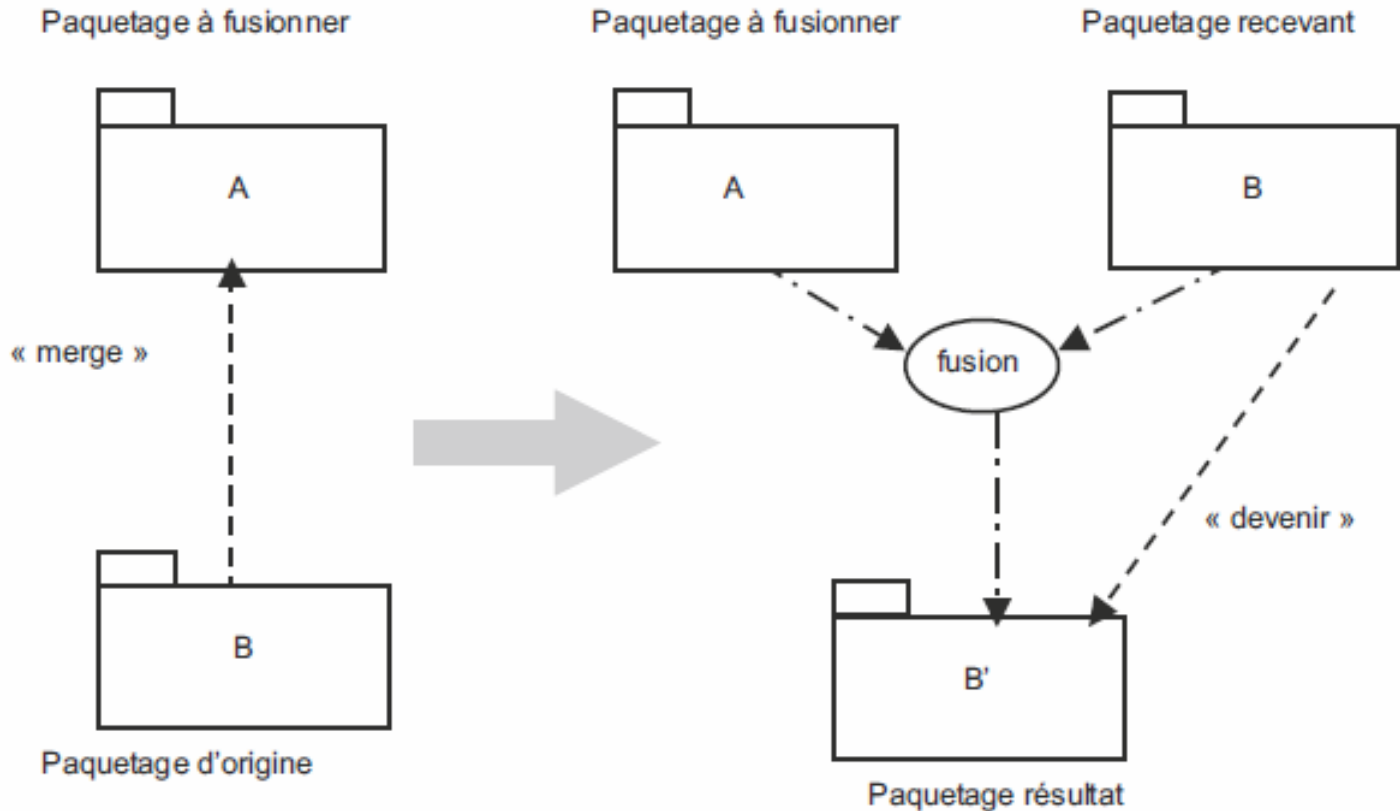
Merge

- This type of link allows **two packages to be merged**, resulting in a package that contains the **combined contents** of the two original packages.
- In this case, the dependency link includes the keyword « **merge** ».
- To clearly specify this operation, it is important to **assign roles** to the packages involved in the merge.
- Three roles can be distinguished:
 - **The package to be merged** – enters the merge but is preserved after the merge.
 - **The receiving package** – the original package before the merge, which is not preserved after the merge.
 - **The resulting package** – contains the result of the merge and **overwrites** the contents of the original receiving package.

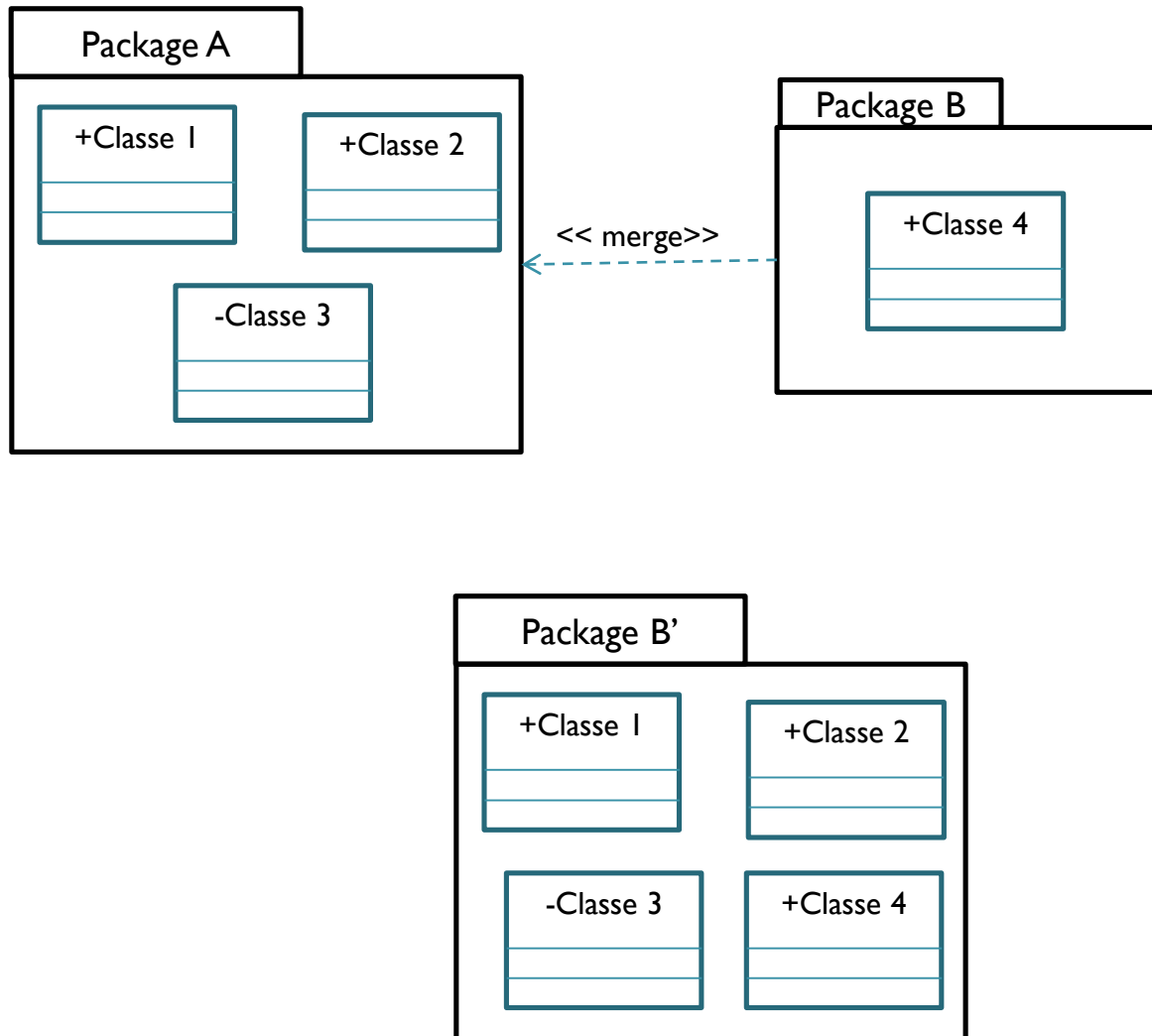
dependancy link « merge ».

- This type of link allows **two packages to be merged**, resulting in a package that contains the **combined contents** of the two original packages.
- In this case, the dependency link includes the keyword « **merge** ».
- To clearly specify this operation, it is important to **assign roles** to the packages involved in the merge.
- Three roles can be distinguished:
 - **Package to be merged** – enters the merge but is **preserved** after the merge.
 - **Receiving package** – the original package **before the merge**, which is **not preserved** after the merge.
 - **Resulting package** – contains the **result of the merge** and **overwrites** the contents of the original receiving package.

Depandancy link « merge ».



Exemple





L3 informatique

Notes de Cours

Génie logiciel

Cours 6

Diagramme de **package**

Présenté par
Mr KHELIFA N.

Introduction

- La visualisation, la construction, la spécification et la documentation **des grands systèmes** implique la manipulation d'un **nombre élevé** de classes, d'interfaces composants, nœuds, diagrammes et autre éléments
- D'où la nécessité de découper ces éléments en groupe
- Le package (paquetage) est un mécanisme générique pour organiser les éléments de modélisation en groupe
- Le découpage en paquetage doit traduire un découpage logique du système à construire qui corresponde à des espaces de nommage homogènes
 - On peut contrôler la visibilité des éléments
 - Pour présenter les différentes vues de l'architecture d'un système

Diagramme de package

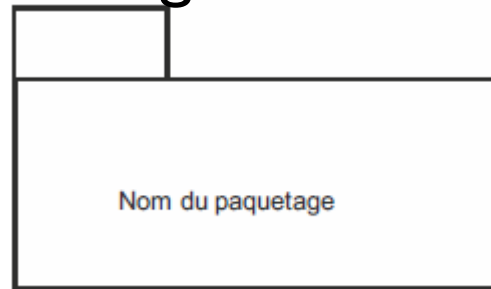
- Est un diagramme structurel, il comprends les éléments suivant:
 - Package
 - Relations entre package (dépendance)

Package

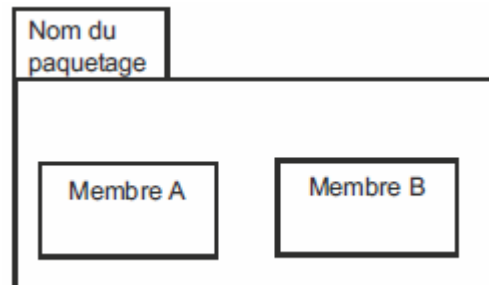
- Définit un espace de nom (namespace)
- Un **paquetage** regroupe des éléments de la modélisation appelés aussi membres (regroupement de classes, diagrammes, fonctions, interfaces...), portant sur un sous-ensemble du système.
- Les éléments d'un paquetage peuvent avoir une visibilité déclarée soit de type public (+) soit privé (-).
- Un paquetage peut importer des éléments d'un autre paquetage.
- Un paquetage peut être fusionné avec un autre paquetage

Représentation

- On peut représenter un package avec trois représentation possible :
- **Représentation globale** – Le nom du paquetage se trouve à l'intérieur du grand rectangle.

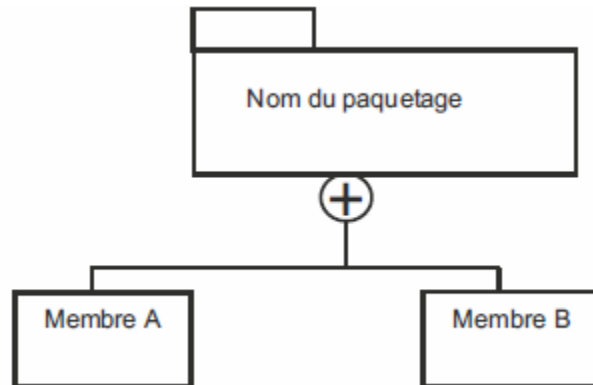


- **Représentation détaillée** – Les membres du paquetage sont représentés et le nom du paquetage d'ensemble s'inscrit dans le petit rectangle.



Représentation

- **Représentation éclatée** – Les membres du paquetage sont reliés par un lien connecté au paquetage par le symbole $\oplus$.

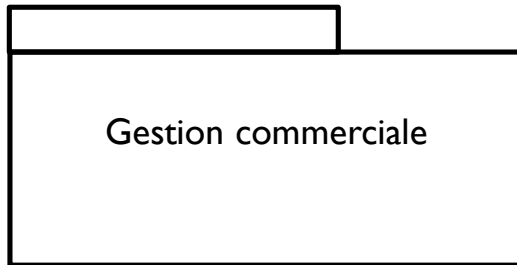


Exemple

- Le package gestion commerciale qui contient deux membres : commande et facture.

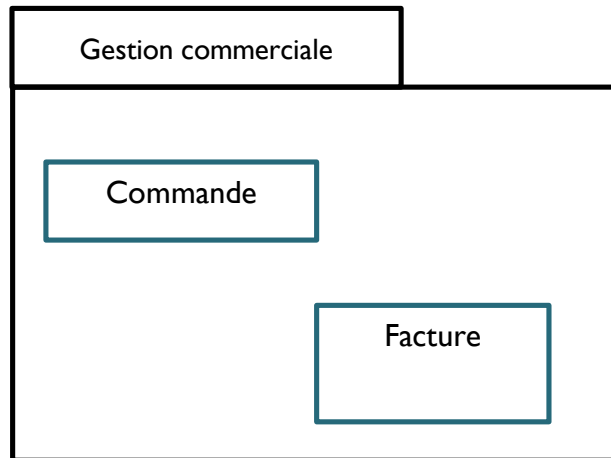
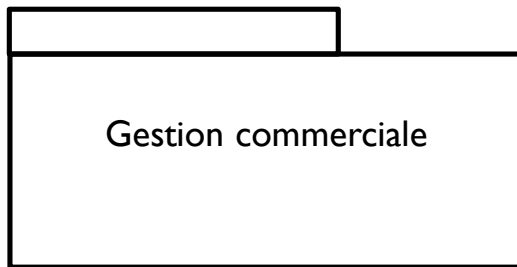
Exemple

- Le package gestion commerciale qui contient deux membres : commande et facture.



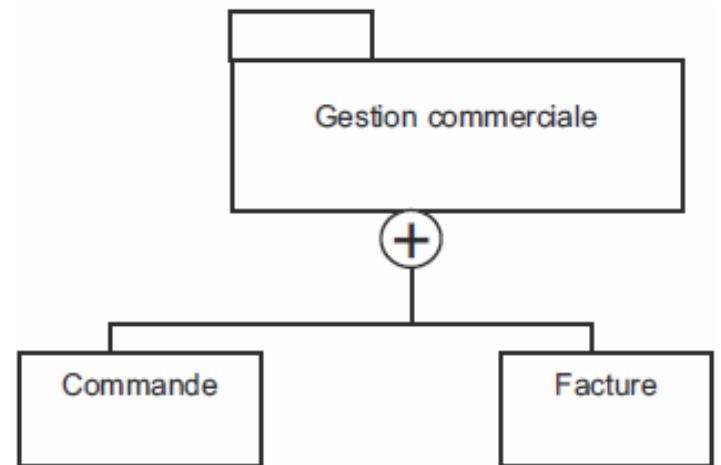
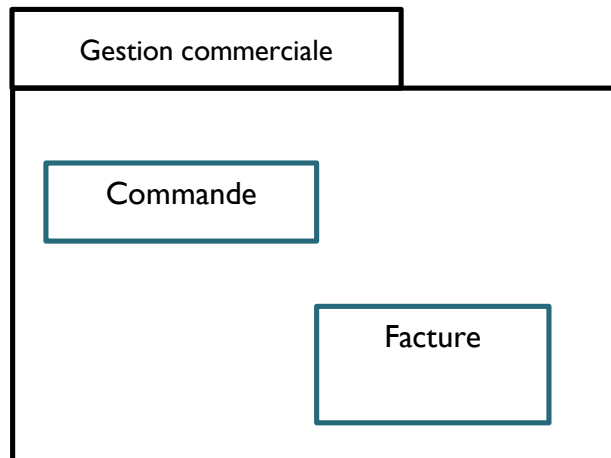
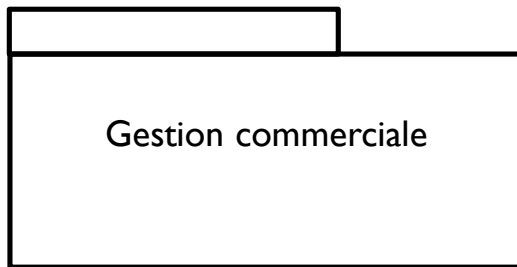
Exemple

- Le package gestion commerciale qui contient deux membres : commande et facture.



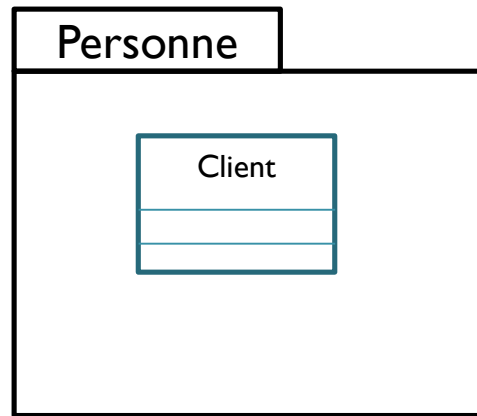
Exemple

- Le package gestion commerciale qui contient deux membres : commande et facture.

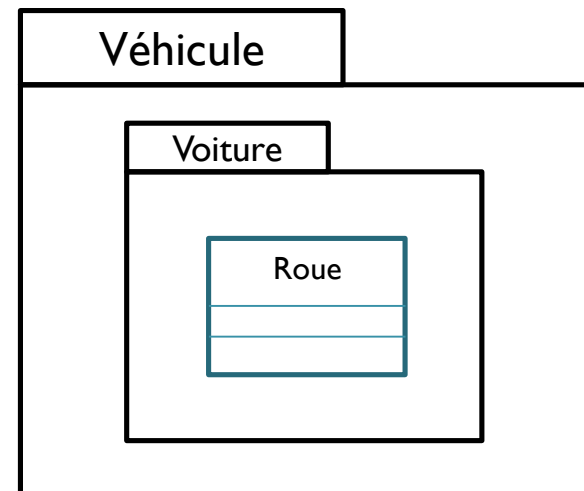


Espaces de nom des membres

Pour faire appel à un élément d'un paquetage, il faut indiquer le nom du paquetage (espace de nommage) suivi de deux fois deux points (::) puis du nom de l'élément.



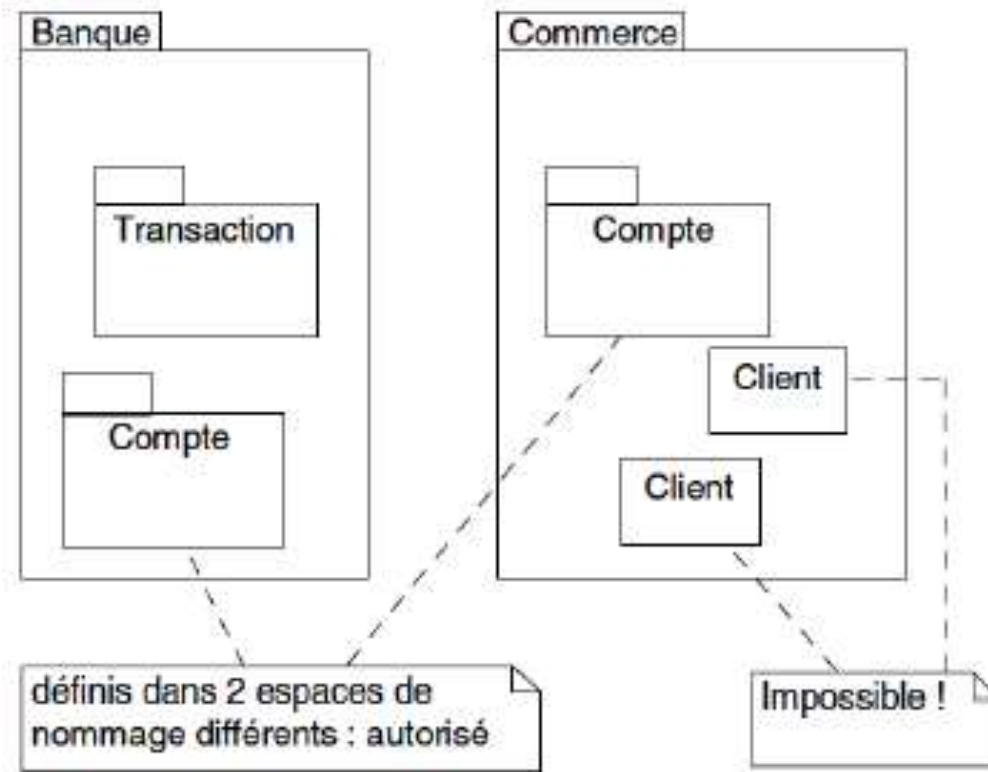
Personne::Client



Véhicule::Voiture::Roue

- **Les paquetages peuvent s'imbriquer (décomposition hiérarchique) mais pas se chevaucher. Un élément du système ne peut appartenir qu'à un et un seul paquetage. Chaque paquetage doit posséder un nom différent**

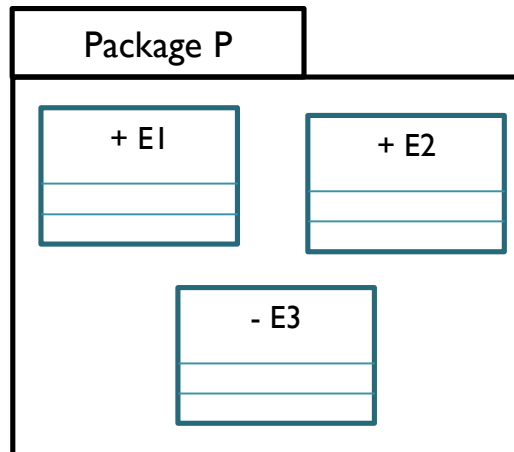
Exemple



- Nom complet = nom préfixé par les noms des paquetages englobants : $\text{Banque::Compte} \neq \text{Commerce::Compte}$

Visibilité

- Visibilité d'un élément E dans un package P :
Public (+) : Élément visible par tous (utilisation du nom complet)
Privé (-) : Élément visible uniquement par
 - Les autres éléments de P
 - Les éléments englobés par E (si E est un package)

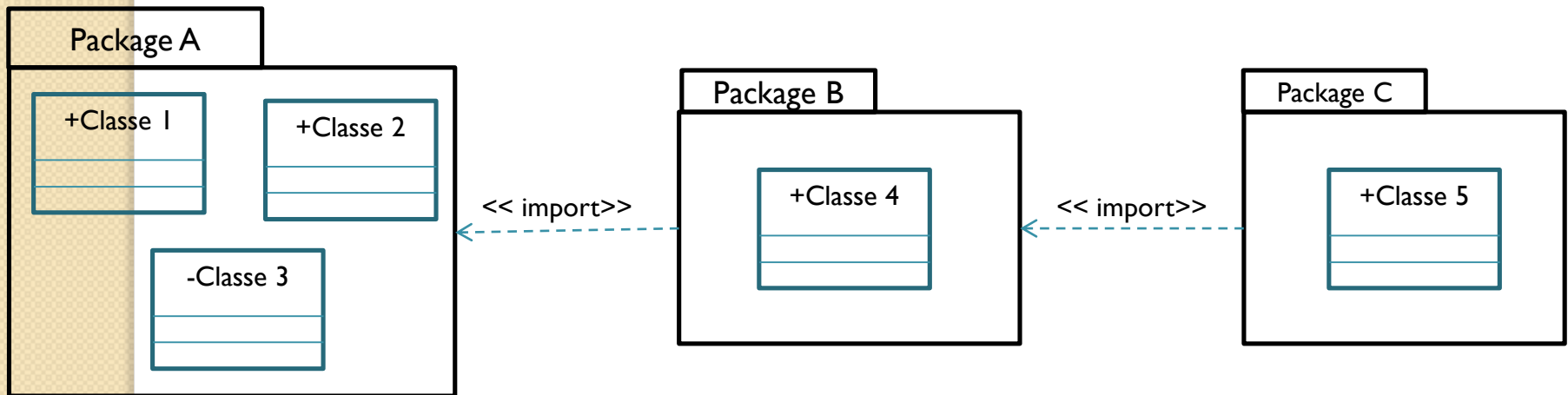


Dépendance entre paquets

- La **dépendance entre paquets** peut être qualifiée par un niveau de visibilité qui est soit public soit privé.
 - Par défaut le type de visibilité est public.
- À chaque type de visibilité est associé un lien de dépendance.
- Les deux types de dépendances entre paquets sont :
 - « **import** »
 - « **access** »

import

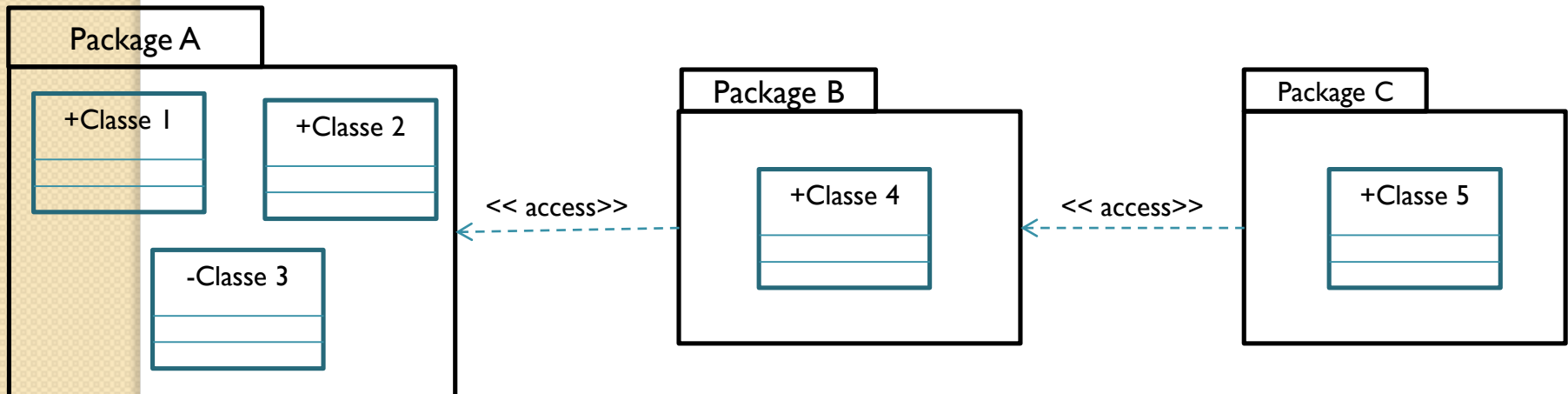
- Ce type de dépendance permet, pour un paquetage donné, d'importer l'espace de nommage d'un autre paquetage.
- tous les membres du paquetage donné ont accès à tous les noms des membres du paquetage importé sans avoir à utiliser explicitement le nom du paquetage concerné.
- Ce type de dépendance correspond à un lien ayant une visibilité « public ».



Le paquetage B importe **Classe 1** et **Classe 2** (pas **Classe 3** qui a une visibilité de type **privée**). **Classe 1** et **Classe 2** ont une visibilité de type **public** dans paquetage B. Le paquetage C importe **Classe 1**, **Classe 2** et **Classe 4**.

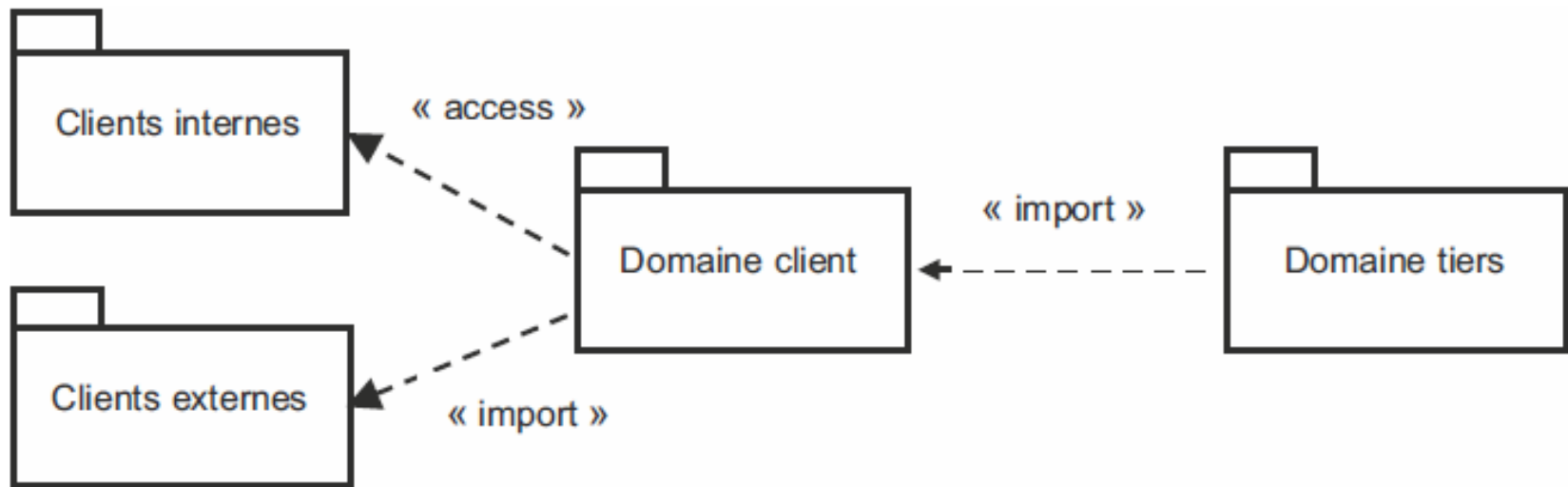
access

- Ce type de dépendance permet, pour un paquetage donné, d'avoir accès à l'espace de nommage d'un paquetage cible.
- L'espace de nommage n'est donc pas importé et ne peut être transmis à d'autres paquetages par transitivité.
- Ce type de dépendance correspond à un lien ayant une visibilité « privé »



Le paquetage B a accès à **Classe 1** et **Classe 2** (pas à **Classe 3** qui a une visibilité de type **privée**). **Classe 1** et **Classe 2** ont une visibilité de type **privé** dans paquetage B. Le paquetage C a accès à **Classe 4** (pas à **Classe 1** et **Classe 2** qui ont une visibilité de type **privée** dans paquetage B).

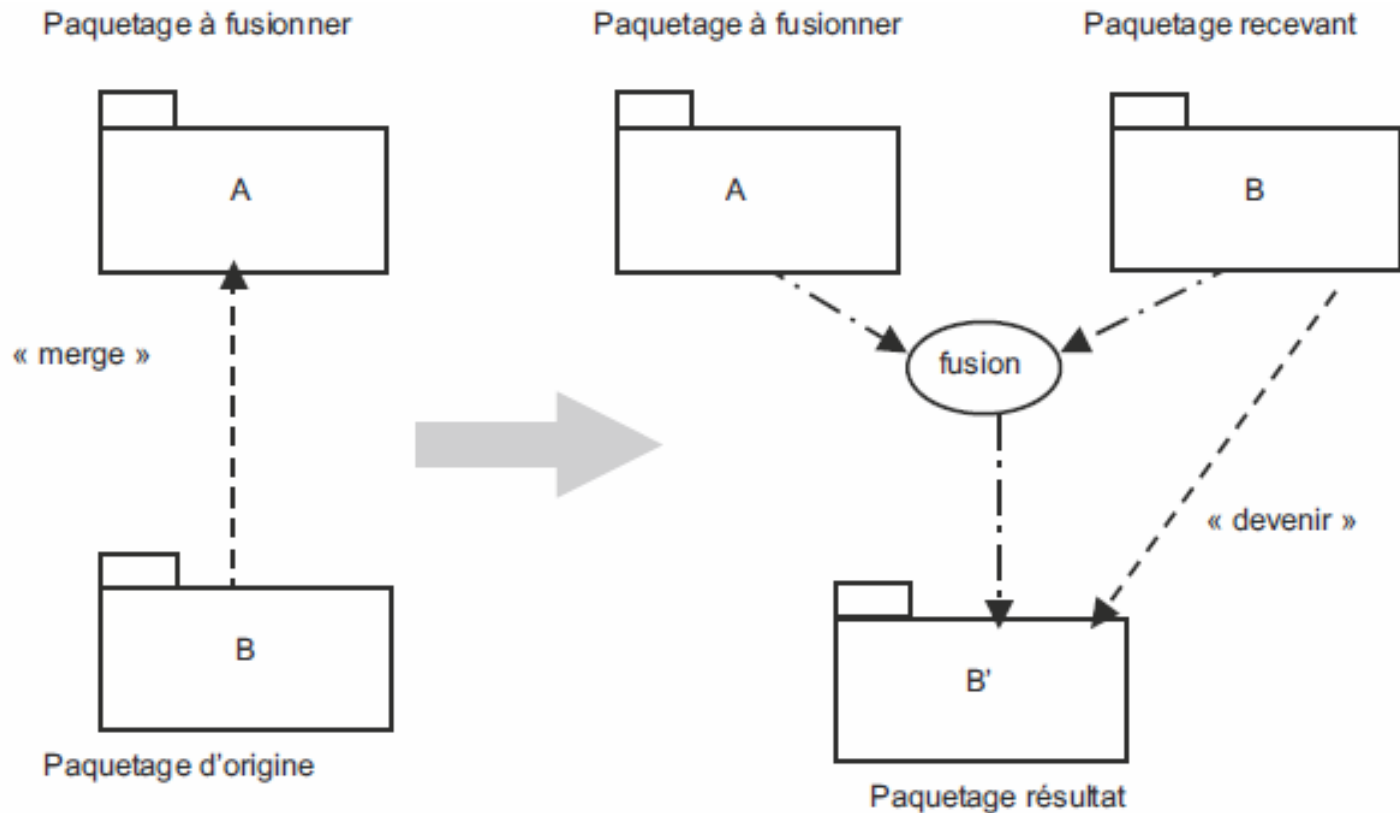
Exemple



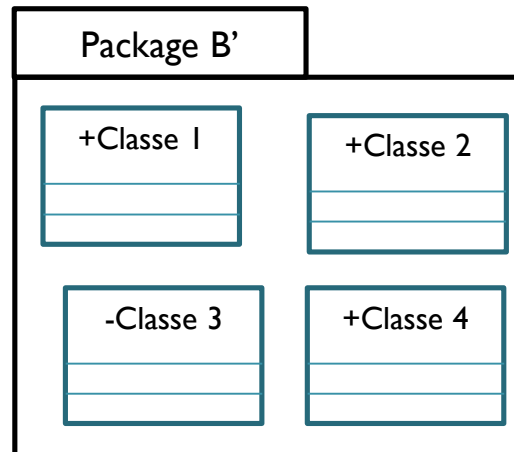
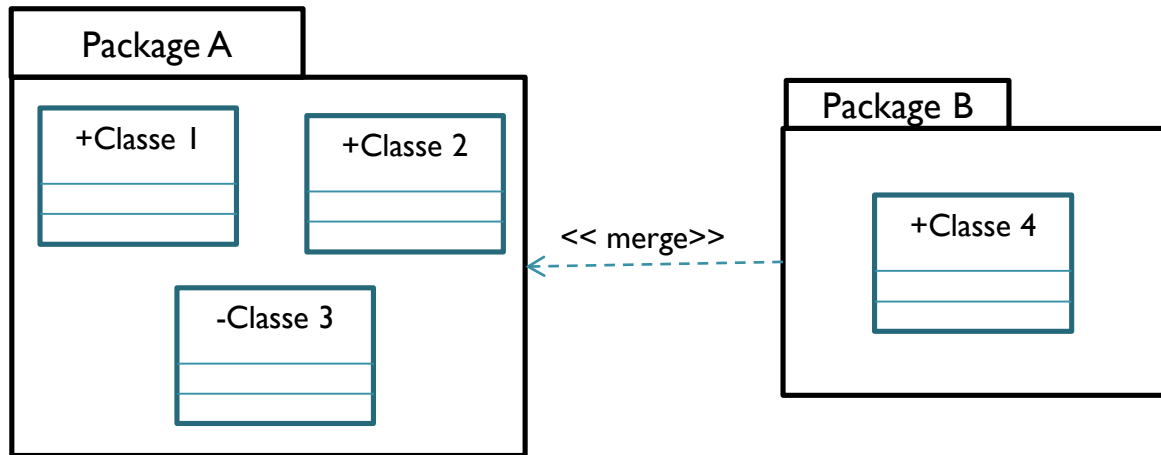
Le lien de dépendance « merge ».

- Ce type de lien permet de fusionner deux paquetages et d'obtenir ainsi un paquetage contenant la fusion des deux paquetages d'origine.
- Le lien de dépendance comporte dans ce cas le mot-clé « merge ».
- Pour bien clarifier cette opération, il est important de qualifier par des rôles les paquetages dans cette fusion.
- Ainsi trois rôles sont à distinguer :
 - le paquetage à fusionner (entrant dans la fusion, mais préservé après la fusion) ;
 - le paquetage recevant (paquetage d'origine avant la fusion, mais non conservé après la fusion) ;
 - le paquetage résultat (paquetage contenant le résultat de la fusion et écrasant le contenu du paquetage d'origine).

Le lien de dépendance « merge ».



Exemple





Merci pour votre attention